📄 Checkpoint 2 – Midterm (Weeks 5–8)

Focus: Prompt Architecture and Vector Indexing

1. Prompt Architecture

FitGen uses three layered prompts (expanded from the original two-prompt plan once implementation began, to add an explicit safety layer for injury-related queries):

System Prompt

Constrains the model's identity and five hard rules before it ever sees a question — ground every answer in retrieved context only, respect all stated user constraints, defer to professionals on red-flag pain symptoms, refuse off-topic questions, and format answers as structured plans.

```
"You are FitGen, a certified-trainer-style fitness assistant. You generate
personalized workout plans and answer exercise questions using ONLY the
exercise and training-principle context provided to you..."
```

User Prompts

Capture goals, duration, equipment, and fitness level, exactly as originally planned — e.g., “I'm a beginner, I have dumbbells only, and I can train 3 days a week. My goal is general strength.”

Context Prompt

Feeds retrieved exercises and recent conversation history into Google's Gemini API (gemini-2.5-flash, via the langchain-google-genai integration) so responses are grounded in the database. This replaces the original plan to use OpenAI — Gemini was selected during implementation for its generous free tier, which better fit the project's budget-conscious framing (see Project Proposal, Problem Statement).

```
"Retrieved context (use this, and only this, as your factual basis):
{context} ... Conversation so far: {chat_history} ... User's new message:
{question}"
```

Safety Refusal Prompt (added during implementation)

A dedicated rule instructing the model to stop and recommend professional care for red-flag pain symptoms (sharp/radiating pain, numbness, swelling, instability) instead of prescribing an exercise. This was added because the original two-prompt plan didn't explicitly cover injury-risk scenarios, which testing showed was the highest-stakes failure mode for a fitness chatbot.

Output Format

Structured plans with days, sets, reps, and progression, exactly as originally planned:

```
**Goal:** ... **Day 1:** Exercise — sets x reps (rest) ... **Day 2:** ...
**Notes:** safety notes or equipment substitutions
```

2. Vector Indexing

Implementation

FAISS, via LangChain's langchain_community.vectorstores.FAISS — as originally planned. Chunks are embedded locally using sentence-transformers/all-MiniLM-L6-v2 (Hugging Face, 384 dimensions) before being added to the index.

Indexing Strategy (revised from original plan)

The original plan called for organizing embeddings by category (strength, cardio, flexibility). During implementation this was changed to a single unified FAISS index covering all 84 chunks (52 exercise entries + 5 training-principle documents). Reasoning: at this dataset size, a single index with top-k similarity search retrieves relevant results across categories without the added complexity of routing a query to the correct sub-index first — and many real queries (e.g., “what's a good warm-up before a strength day”) legitimately span more than one category. Category-based indexing remains a reasonable direction to revisit if the dataset grows substantially (see Model Customization Note / Framework Comparison Note for related scaling discussion).

Optimization

Top-k similarity search (k=4) returns the most relevant chunks per query. An empirical comparison of Cosine Similarity, Euclidean Distance, and Dot Product on the actual dataset showed all three metrics produce identical rankings, because the embeddings are unit-normalized. Cosine similarity is used conceptually, implemented efficiently via FAISS inner-product search over the normalized vectors (see Distance Metric Comparison document for the full experiment and results).

Example (as actually implemented)

User says “my knees hurt during squats, what should I do” → vector search retrieves the injury-safety reference on knee-friendly squat regressions plus relevant dumbbell/bodyweight leg exercises → chatbot generates a response recommending a regression (e.g., goblet squat, reduced range of motion) instead of prescribing the original movement, citing the retrieved safety guidance.

Original example retained for reference: User says “I want a 3-day strength plan” → vector search retrieves relevant strength-rep-range guidance and compound exercises (e.g., squat, deadlift variant, bench/press variant) → chatbot generates a natural workout plan grounded in that retrieved content.